PRÁCTICA EXTRA: LIBRERÍAS EN PYTHON

Módulos Estándar: os, sys y platform — Ejercicios Complementarios

Objetivo de la Práctica: Reforzar el uso de módulos estándar combinándolos con lo visto en U4 (funciones) y U5 (bucles, slicing, listas).

Estilo y Buenas Prácticas (PEP 8): Usar convención *snake_case* para variables y funciones, *SCREAMING_SNAKE_CASE* para constantes e importar las librerías al inicio del archivo.

Metodología Polya: Identificar la función de la librería requerida, planificar el script y verificar el resultado por pantalla.

SECUENCIA DE PROBLEMAS PRÁCTICOS

PROBLEMA 1: Rutas y carpetas (módulo `os`)

El administrador necesita saber en qué carpeta está trabajando y si existen ciertas rutas.

Consigna:

a) Imprimir la ruta actual con `os.getcwd()`.

b) Imprimir solo el *nombre* de la carpeta actual con `os.path.basename()`.

c) Recorrer con un `for` las rutas `'/home'` y `'/carpeta_que_no_existe'` e informar, para cada una, si no existe, si existe y es directorio (`os.path.isdir`), o si existe pero no es directorio.

Fase Polya / Guía de Resolución:

1. ¿qué devuelve `os.path.basename('/a/b/c')`?
2. ¿Qué diferencia hay entre `exists` e `isdir`?
3. Espacio para boceto de código:

PROBLEMA 2: Datos del intérprete (módulo `sys`)

Consigna:

a) Mostrar la versión de Python como `3.x` usando `sys.version_info[0]` y `sys.version_info[1]`.

b) Mostrar la plataforma con `sys.platform`.

c) Convertir `sys.version` en lista con `.split()`, mostrar cuántas palabras tiene y cuál es la primera.

d) Con un `if`, informar si la versión mayor es 3 o más ("compatible") o menor ("demasiado antigua").

Fase Polya / Guía de Resolución:

1. ¿qué tipo de dato es `sys.version_info`?
2. ¿Se puede indexar como una lista?
3. Espacio para boceto de código:

PROBLEMA 3: Informe del sistema (módulo `platform`)

Consigna:
a) Mostrar sistema operativo (`platform.system()`), nombre del equipo (`platform.node()`), arquitectura (`platform.machine()`) y versión de Python (`platform.python_version()`).
b) Con `if/elif/else`, imprimir un mensaje distinto para `"Linux"`, `"Windows"` y cualquier otro sistema.
c) Recorrer `platform.uname()` con un `for` e imprimir cada campo con su etiqueta (Sistema, Nodo, Release, Versión, Máquina) en dos columnas.

Fase Polya / Guía de Resolución:

1. ¿cuántos elementos tiene la tupla de `uname()`?
2. ¿Cómo alinear texto a la izquierda en un f-string (`{texto:<10}`)?
3. Espacio para boceto de código:

PROBLEMA 4: Contar archivos de un tipo (`os` + función + `for`)

Consigna:
a) Definir `contar_por_extension(lista_archivos, extension)` que recorra la lista con un `for`, cuente los nombres que terminan con la extensión indicada (usar `nombre.endswith(extension)`) y devuelva el total con `return`.
b) En el programa principal: obtener la ruta con `os.getcwd()`, listar con `os.listdir()`, llamar a la función con la constante `EXTENSION = ".txt"` e imprimir la ruta, el total de elementos y la cantidad de `.txt`.
c) Probar la función con la lista fija `["a.txt", "b.py", "c.txt", "notas.TXT", "txt"]`. El resultado esperado es 2. ¿Por qué `notas.TXT` no cuenta?

Fase Polya / Guía de Resolución:

1. ¿en qué se parece `endswith` al rebanado `nombre[-4:]`?
2. Espacio para boceto de código:

PROBLEMA 5: Argumentos por línea de comandos (`sys.argv`)

Ejecutar el script así: `python3 problema_5_argumentos_sys_argv.py 4 10 7`
Consigna:
a) Guardar en `argumentos` todos los valores recibidos, **sin** el nombre del script (`sys.argv[1:]`).
b) Si no se recibió ninguno, imprimir un mensaje de uso.
c) Si se recibieron, convertir cada uno a `int` dentro de un `for`, acumular la suma e imprimir cantidad, suma y promedio.

Fase Polya / Guía de Resolución:

1. ¿qué contiene `sys.argv[0]`?
2. ¿Qué tipo de dato tienen los elementos de `sys.argv`?
3. Espacio para boceto de código:

PROBLEMA 6: Integrador, resumen de archivos por tipo (`os` + `platform`)

Consigna:

a) Definir `clasificar(nombre)` que use `os.path.splitext(nombre)[1]` y devuelva `"python"` (`.py`), `"texto"` (`.txt`) u `"otro"`.

b) Definir `resumir(lista_nombres)` que use `clasificar()` dentro de un `for` y devuelva tres valores: cantidad de python, de texto y de otros.

c) En el programa principal: construir con un `for` una lista con **solo los archivos** de la carpeta actual (descartar subcarpetas con `os.path.isfile(os.path.join(ruta, nombre))`), llamar a `resumir()` e imprimir el equipo (`platform.node()`), el sistema (`platform.system()`), la carpeta y los totales.

Fase Polya / Guía de Resolución:

1. ¿cómo devuelve una función más de un valor?
2. ¿Por qué conviene `os.path.join` en vez de concatenar con `"/"`?
3. Espacio para boceto de código: